

AudialMind

Assignment 1

Signal Processing & Audio Classification

Navneet Kumar

25b0305

June 2026

Contents

1	RAVDESS Dataset Analysis	2
1.1	Filename Decoding	2
1.2	Dataset Statistics	2
1.3	Why UAR Over Accuracy	4
2	Acoustic Feature Extraction with openSMILE	6
2.1	Single-File Extraction	6
2.2	Full Dataset Extraction	6
2.3	Feature-Level Analysis	6
3	Classical ML Baselines	9
3.1	SVM Baseline	9
3.2	Random Forest Baseline	11
3.3	Model Comparison	13
4	Feature Space Visualisation with UMAP	14
4.1	2D Projection	14
4.2	Standardised vs Raw Features	16
5	CNN Spectrogram Classifier	17
5.1	Data Pipeline	17
5.2	Model Architecture	17
5.3	Training & Evaluation	18
	Bonus: Custom openSMILE Config File	20

Task 1 RAVDESS Dataset Analysis

Part 1.1 Filename Decoding

The RAVDESS dataset follows a structured 7-part naming convention. Each `.wav` filename looks like this:

03-01-05-01-01-01-24.wav

Each field encodes a specific piece of metadata:

Position	Field	Code	Decoded
1	Modality	03	audio-only
2	Vocal Channel	01	speech
3	Emotion	05	angry
4	Intensity	01	normal
5	Statement	01	“Kids are talking by the door”
6	Repetition	01	1st repetition
7	Actor	24	Actor #24

I wrote a function `parse_ravdess_filename(path: str) -> dict` that takes any RAVDESS filepath, splits the basename by hyphens, and maps each code to its human-readable label using lookup dictionaries. The function returns a dictionary with keys: `modality`, `vocal_channel`, `emotion`, `intensity`, `statement`, `repetition`, and `actor`.

The emotion codes map as follows: 01=neutral, 02=calm, 03=happy, 04=sad, 05=angry, 06=fearful, 07=disgust, 08=surprised.

Part 1.2 Dataset Statistics

Class Distribution

After loading the full speech subset (modality = audio-only), I found **1440 files** total. Seven out of eight emotions have 192 files each, but **neutral has only 96** — exactly half. I think, this is because neutral has no intensity like angry and surprised ones have. This creates a 2:1 imbalance ratio.

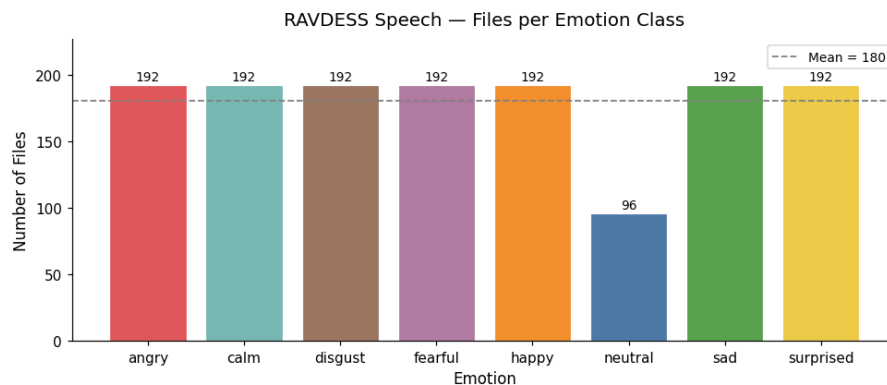


Figure 1: Number of files per emotion class. Neutral has half the samples compared to every other class.

This imbalance is not extreme, but it is enough to bias accuracy-based metrics, which is why we need UAR (discussed in Part C).

Audio Duration

Statistic	Value
Mean duration	3.701 s
Std deviation	0.337 s
Minimum	2.936 s
Maximum	5.272 s

Most recordings are tightly clustered around 3.5–4 seconds, which makes sense since all actors spoke the same two short sentences. The histogram below confirms this — the bulk of files sit between 3.3 and 4.0 seconds, with a few outliers stretching to 5.3 seconds.

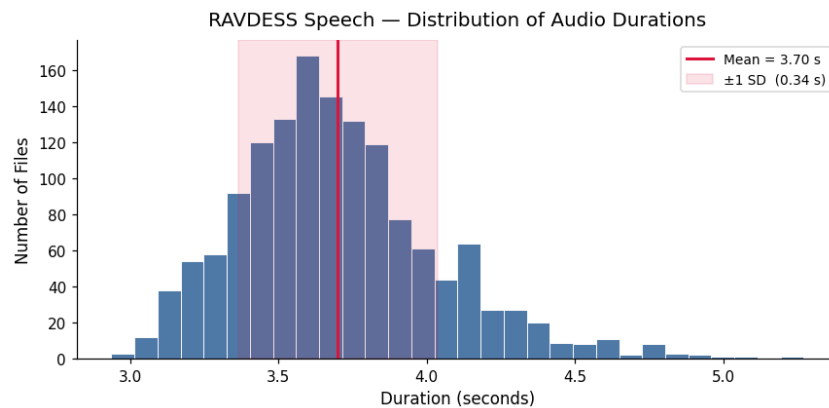


Figure 2: Distribution of audio durations across all 1440 speech files.

Waveforms and Mel-Spectrograms

I picked one happy, one sad, and one angry file and plotted their waveforms (top row) and mel-spectrograms (bottom row) side by side.

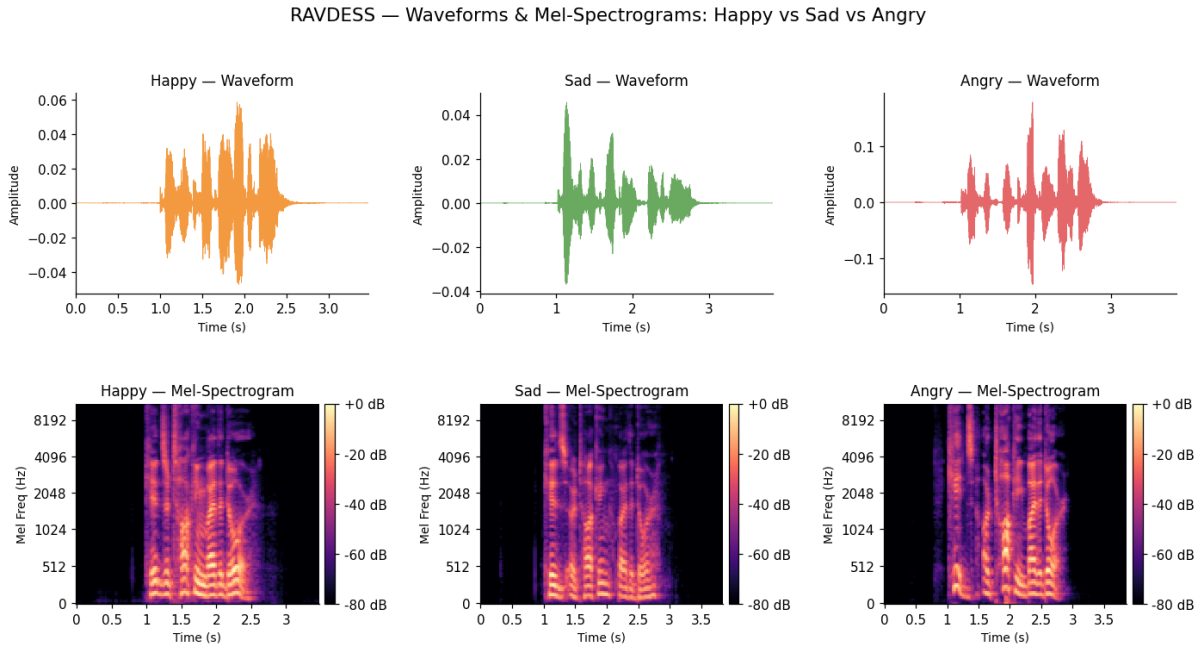


Figure 3: Waveforms (top) and mel-spectrograms (bottom) for happy, sad, and angry speech.

A few things stand out visually:

- **Happy** has moderate amplitude with noticeable energy spread across a wider frequency range.
- **Sad** looks quieter overall — lower amplitude in the waveform and less energy in the higher frequency bands of the spectrogram. It looks “flatter” and more monotone.
- **Angry** has the most intense waveform with sharp peaks and the richest spectrogram — lots of energy across all frequencies, which corresponds to the louder, more forceful way people speak when they are angry.

These visual patterns make intuitive sense — angry speech is loud and intense, sad speech is soft and subdued.

Part 1.3 Why UAR Over Accuracy

I’ll explain this thing with our dataset example. In our dataset, neutral has 96 samples while every other emotion has 192. Now imagine a lazy model that just never predicts neutral — it completely ignores that class. Because neutral is only about 6.7% of the data, this model can still score around 84% accuracy. That sounds decent, but it is actually terrible for neutral speakers.

UAR fixes this problem. Instead of counting how many total predictions are correct (which lets the majority classes dominate), UAR computes the recall of each class separately and then averages them equally:

$$\text{UAR} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k$$

where K is the number of classes. Each class gets exactly $\frac{1}{K}$ weight, regardless of how many samples it has.

I actually tested this in the notebook — I simulated a biased model that never predicts neutral. It scored 84.0% accuracy but only 78.8% UAR. That 5.3 percentage point gap is exactly the kind of thing UAR is designed to catch. Accuracy was giving the model credit it did not deserve, while UAR correctly penalised it for completely failing on neutral.

So in any situation where classes are not perfectly balanced (which is almost always the case with real emotion data), UAR is the metric we should trust. UAR helps minority data of specific distribution a space to represent themselves.

Task 2 Acoustic Feature Extraction with openSMILE

Part 2.1 Single-File Extraction

Using the `opensmile` Python wrapper with the `eGeMAPSv02` feature set at the `Functionals` level, I extracted features from a single RAVDESS file. The output shape was `(1, 88)` — one row with 88 features.

The 88 features cover pitch (F0), loudness, spectral properties, MFCCs, jitter, shimmer, and harmonics-to-noise ratio, with various statistical functionals (mean, standard deviation, percentiles, slopes) applied to each. The output is given below:

<https://docs.google.com/spreadsheets/d/1LwFKaLGS4bYJVpKpNZXvUxJx3VWph9g3EELrVIXTVcG/edit?usp=sharing>

Three Features Explained

1. **F0semitoneFrom27.5Hz_sma3nz_amean** — This is the average pitch of the voice, measured in semitones. It tells us how high or low someone's voice sounds over the whole recording. Higher values mean a higher-pitched voice.
2. **loudness_sma3_amean** — This captures the average perceived loudness of the speech. It is not just raw amplitude — it is modelled to approximate how loud we as humans would actually hear it. Angry speech tends to have high values here.
3. **mfcc1_sma3_amean** — The first MFCC (Mel-Frequency Cepstral Coefficient) averaged over the whole file. MFCCs represent the overall shape of the frequency spectrum in a compact way. The first coefficient basically captures the overall energy distribution across frequencies.

Part 2.2 Full Dataset Extraction

I wrote a loop that goes through all 1440 speech files, runs `smile.process_file()` on each one, parses the emotion label from the filename, and collects everything into a single `DataFrame`. The whole thing took about 5 minutes with a `tqdm` progress bar.

Item	Value
Files extracted	1440
Feature columns	88
Label column	1
Final <code>DataFrame</code>	1440×89
Output file	<code>ravdess_egemaps.csv</code>

Part 2.3 Feature-Level Analysis

Per-Class Pitch

I computed the mean of `F0semitoneFrom27.5Hz_sma3nz_amean` for each emotion class.

Emotion	Mean Pitch (semitones)
calm	30.03
neutral	30.28
disgust	32.07
sad	32.59
happy	35.84
surprised	35.93
angry	36.54
fearful	37.93

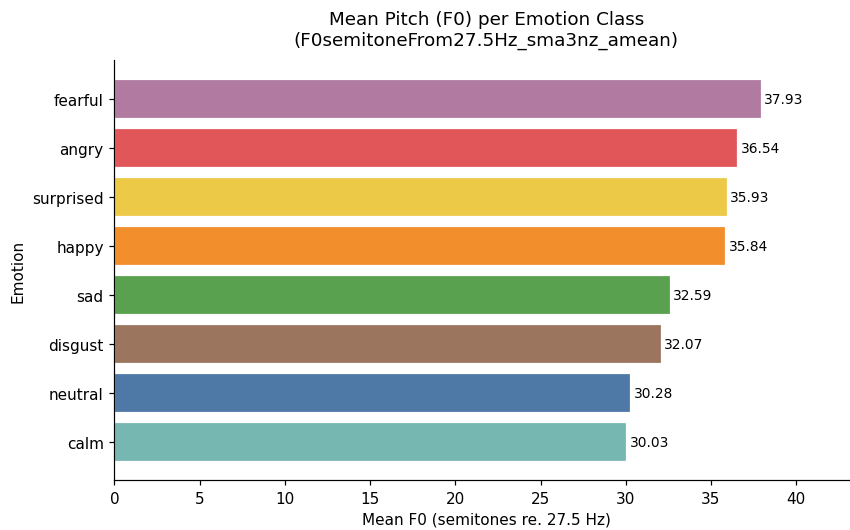


Figure 4: Per-class mean pitch (F0 in semitones). Fearful has the highest pitch.

Fearful has the highest mean pitch, which makes a lot of sense. When someone is scared, their body tenses up — including the vocal folds — which naturally raises the voice pitch. Same thing happens with surprise and anger. On the other end, calm and neutral speech is relaxed and low-pitched, which is exactly what we see here.

This ordering follows the arousal dimension of emotion: high-arousal emotions (fear, anger, surprise, happiness) push pitch up, while low-arousal ones (calm, neutral, sadness) keep it low.

Feature Correlation

I computed the full 88×88 correlation matrix.

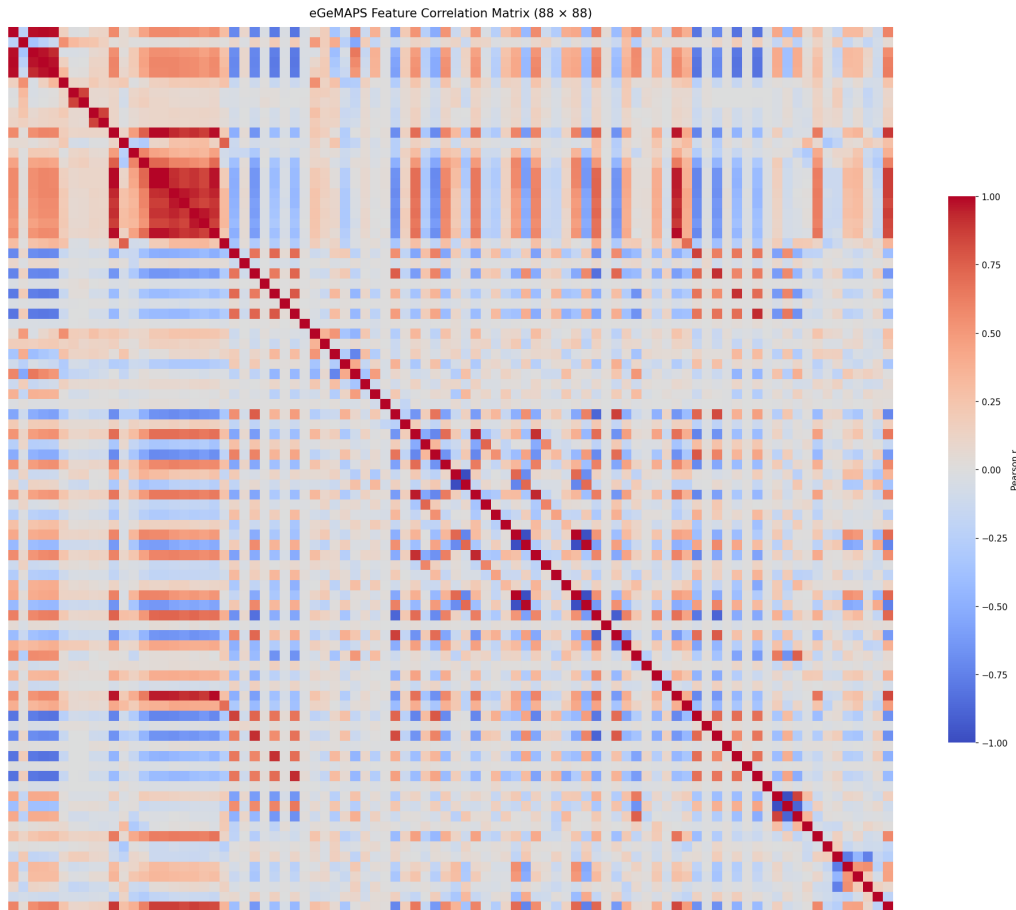


Figure 5: Correlation heatmap of all 88 eGeMAPS features. Bright red/blue patches indicate highly correlated feature groups.

There are **45 feature pairs** with $|r| > 0.9$ which are given here: <https://docs.google.com/spreadsheets/d/1iczollz3tLNYVPg2S9ScRcE7sDmfbVvsRBsnSNpTsug/edit?usp=sharing>
Two notable examples:

- **loudness_sma3_percentile80.0** and **loudness_sma3_pctlrange0-2** ($r = 0.9997$)
— These are both derived from the same loudness contour, just summarised slightly differently. It is no surprise they move together almost perfectly.
- **F2amplitudeLogRelF0_sma3nz_amean** and **F3amplitudeLogRelF0_sma3nz_amean** ($r = 0.997$) — The amplitudes of the second and third formants tend to rise and fall together because they come from the same vocal tract resonance structure.

This level of redundancy is something to keep in mind — many features are essentially telling us the same thing, which means a model using all 88 is not really getting 88 independent pieces of information.

Task 3 Classical ML Baselines

Part 3.1 SVM Baseline

Why Standardise?

SVMs with an RBF kernel work by measuring distances between data points. If one feature is in the range 0–50 (like pitch in semitones) and another is in the range 0–0.01 (like some spectral ratio), the SVM would basically ignore the small-scale feature because it barely affects the distance. StandardScaler fixes this by centering each feature to mean 0 and scaling it to standard deviation 1, so every feature gets a fair say in the distance computation.

10-Fold Stratified CV Results (C=1)

Fold	1	2	3	4
Acc	0.5417	0.6250	0.5694	0.6111
UAR	0.5280	0.6059	0.5461	0.6039
Fold	5	6	7	8
Acc	0.6597	0.5903	0.6250	0.5833
UAR	0.6388	0.5664	0.6261	0.5507
Fold	9	10	Mean	Std
Acc	0.6111	0.5694	0.5986	0.0327
UAR	0.6069	0.5472	0.5820	0.0367

Confusion Matrix

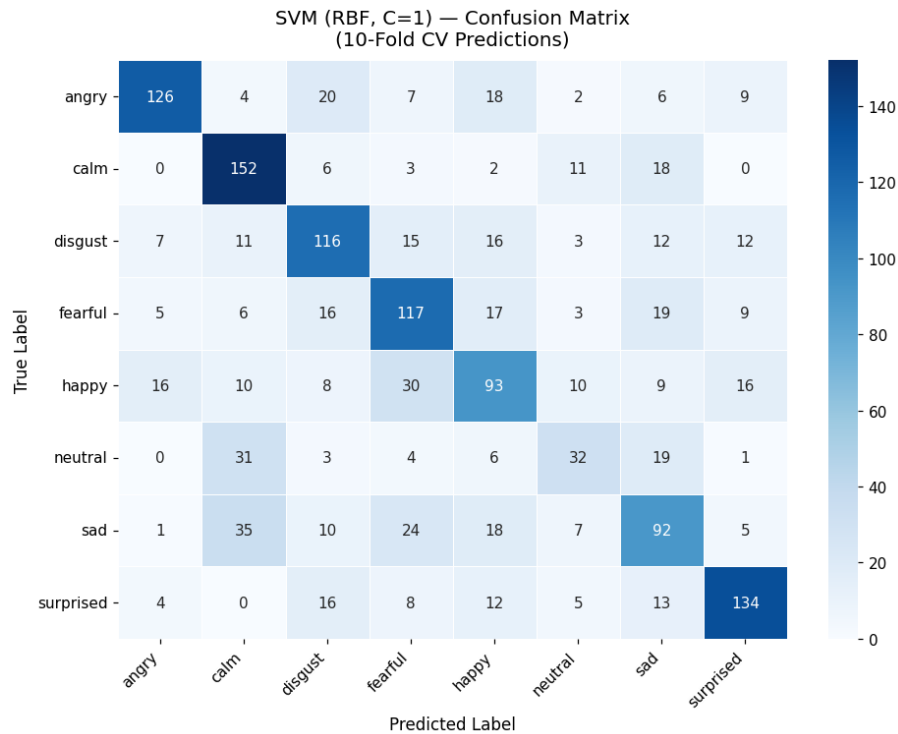


Figure 6: Confusion matrix from cross-validated SVM predictions. The most frequent misclassifications lie along the “arousal axis.”

The most confused pairs:

True	Predicted As	Count
sad	calm	35
neutral	calm	31
happy	fearful	30
sad	fearful	24
angry	disgust	20

There is a clear pattern here — the confusion follows an **arousal axis**. Low-arousal emotions (sad, neutral, calm) are getting mixed up with each other because they all sound soft, slow, and low-pitched. Similarly, high-arousal emotions (happy, fearful) overlap because they both involve raised pitch and faster speech. This is actually a great challenge in speech emotion recognition — the acoustic difference between calm and neutral is genuinely small, even for humans.

Effect of Regularisation Parameter C

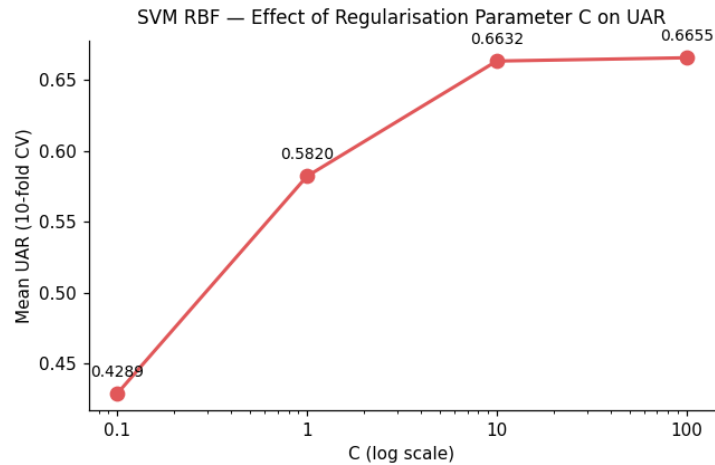


Figure 7: Mean UAR across 10 folds for different values of C.

C	Mean UAR
0.1	0.4289
1	0.5820
10	0.6632
100	0.6655

C controls how much the SVM cares about getting every training point right vs keeping the decision boundary smooth. At $C=0.1$, it is too relaxed — it allows too many misclassifications for the sake of a wide margin, so it underfits. As we increase C, the model gets stricter and performance improves significantly. From $C=10$ to $C=100$ the improvement is marginal (0.6632 to 0.6655), so $C=10$ is the sweet spot — going higher just adds computation without meaningful gain.

Part 3.2 Random Forest Baseline

Results

Random Forest with 200 trees achieved:

$$\text{Mean Accuracy} = 0.6354 \pm 0.0342 \quad \text{Mean UAR} = 0.6240 \pm 0.0369$$

Feature Importances

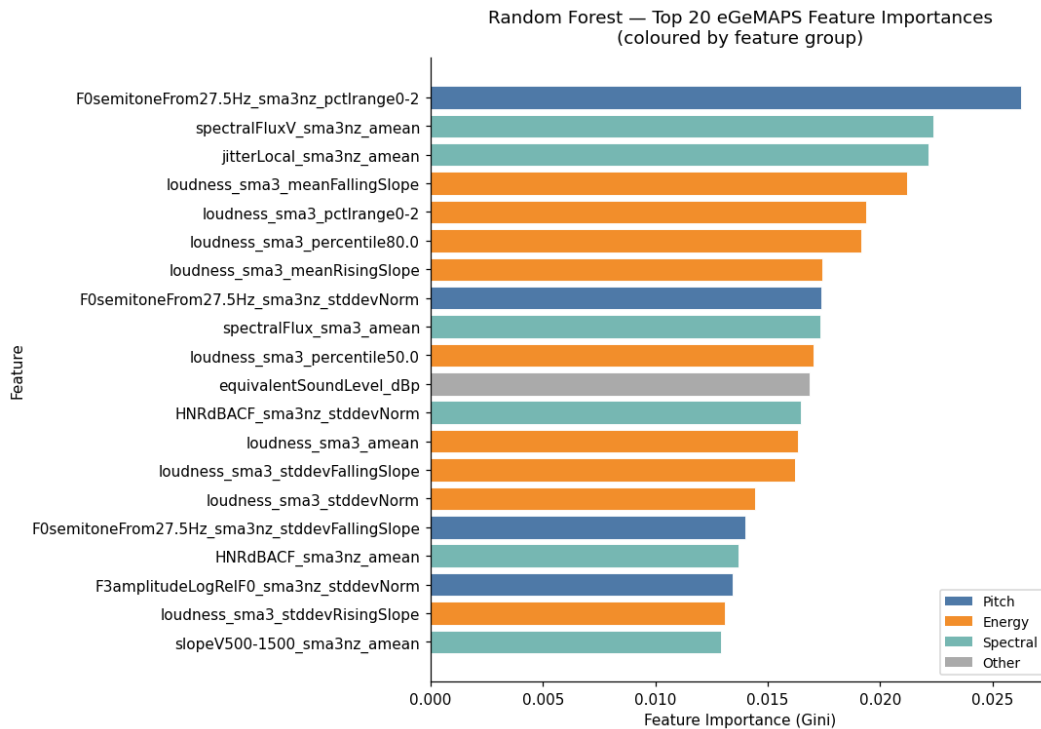


Figure 8: Top 20 most important eGeMAPS features from Random Forest, coloured by feature group.

The most important feature overall is **F0 pitch range** (pctlrage), followed by **spectral flux** (voiced) and **jitter**. Looking at the colour groupings:

- **Pitch features** dominate — pitch range and pitch variability are the strongest signals. This makes sense because different emotions produce very different pitch patterns (as we saw in Task 2).
- **Energy/loudness features** are next — loudness slopes and percentiles capture how dynamically someone speaks. Angry speech has sharp loudness changes, calm speech is steady.
- **Spectral features** like spectral flux capture how quickly the frequency content changes frame-to-frame — more dynamic in high-arousal emotions.

This is consistent with what we learn about speech science: the main acoustics involved with emotional expression are pitch variation and intensity variation. It's comforting that the Random Forest was able to discover this correlation for itself — it shows the features are doing their job and the model isn't just being fooled by noise.

Part 3.3 Model Comparison

Model	Features	Mean Acc.	Mean UAR	UAR Std
SVM (RBF, C=1)	eGeMAPS	0.5986	0.5820	0.0367
SVM (RBF, C=10)	eGeMAPS	—	0.6632	—
Random Forest	eGeMAPS	0.6354	0.6240	0.0369

Table 1: Comparison of classical baselines on RAVDESS eGeMAPS features.

With default settings (i.e., C=1), however, Random Forest significantly outperforms SVM in this case (0.6240 vs. 0.5820 UAR). But with the optimal value of C set to 10, SVM surpasses Random Forest, achieving 0.6632 UAR. The conclusion about which classifier is the most suitable for use in this dataset is therefore conditional and dependent upon specific circumstances – if ease of use and minimal optimization is preferred, Random Forest will serve this purpose better. Otherwise, SVM with Gaussian kernel and optimized C will produce the highest score.

For the AudialMind project baseline, I would go with **SVM (C=10)** since the tuning effort is minimal (just one parameter) and the payoff is significant.

Task 4 Feature Space Visualisation with UMAP

Part 4.1 2D Projection

UMAP Scatter Plot (n_neighbors=15)

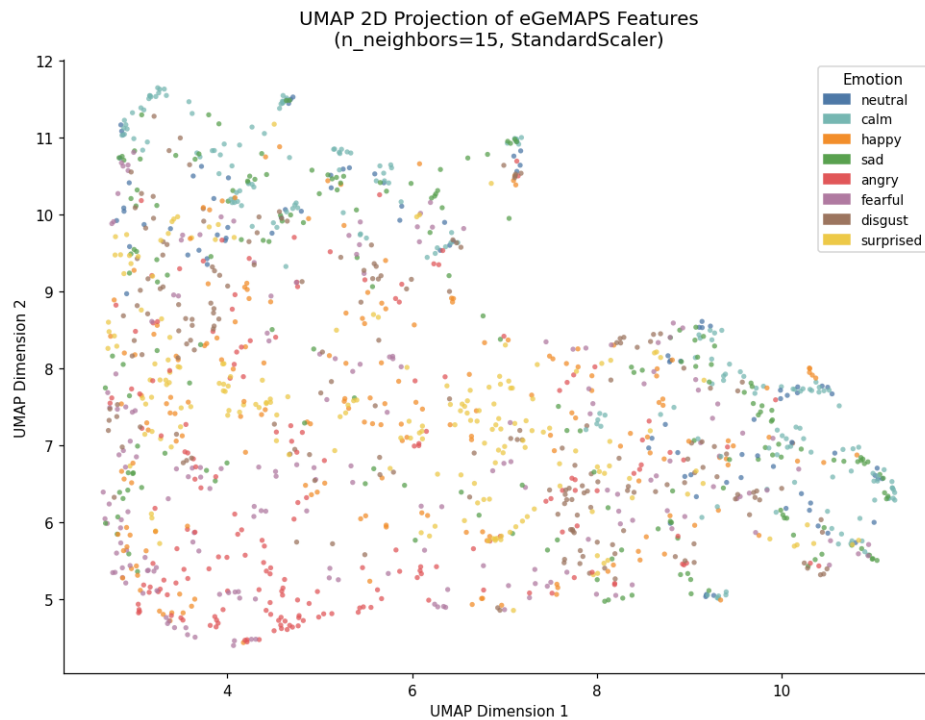


Figure 9: UMAP 2D projection of eGeMAPS features, coloured by emotion class (n_neighbors=15).

Cluster Structure

The first thing that stands out is that there are no clean, well-separated clusters. The plot shows one large continuous arc/banana shape, which tells us that **emotions are not cleanly separable** in the 88-dimensional eGeMAPS space.

That said, some structure is visible:

- **Angry (red)** is the most distinct — it concentrates at the bottom-left corner of the arc, somewhat separated from the rest.
- **Neutral (blue)** tends to shift toward the right side of the plot.
- **Calm, sad, happy, disgust** are heavily overlapping in the central region — the model has a hard time telling these apart, which is consistent with the confusion matrix we saw in Task 3.

This visualisation helps explain why our classifiers top out around 66% UAR — the feature space just does not separate all eight emotions cleanly.

Effect of `n_neighbors`

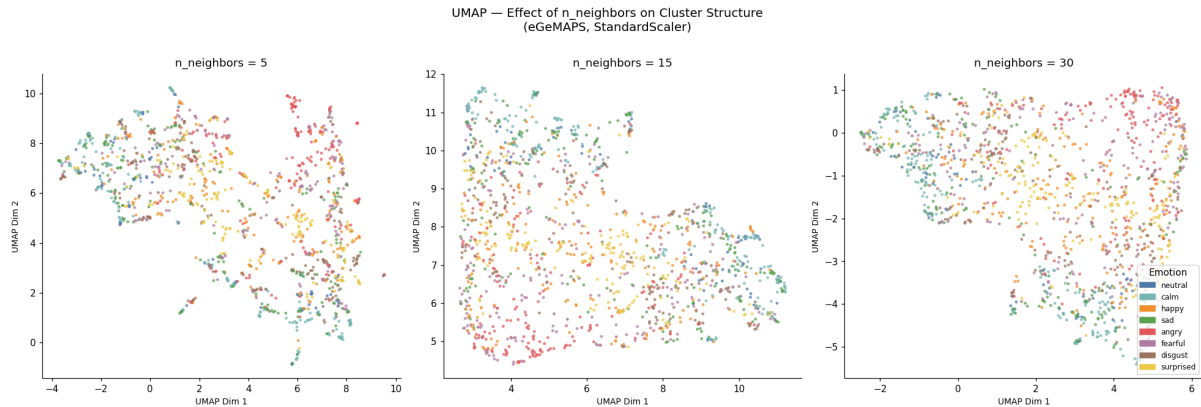


Figure 10: UMAP projections with different `n_neighbors` values: 5, 15, and 30.

- **`n_neighbors=5` (left):** Way too local. The plot is fragmented into tiny islands with lots of isolated points. There is no meaningful global structure — it is all noise and local quirks of individual recordings.
- **`n_neighbors=15` (middle):** The sweet spot. The arc shape is smooth and clear, angry is visibly separated, and you can see the gradual transition between emotion groups. This gives the best balance between preserving local neighbourhoods and showing the overall layout.
- **`n_neighbors=30` (right):** Over-smoothed. The clusters that were barely visible at `n=15` merge even further. The global shape is very smooth but fine-grained structure is lost.

The hyperparameter `n_neighbors` determines whether UMAP should be more locally or globally aware. Low values make UMAP more sensitive to small neighborhoods (helpful for dense clusters, not ideal for noisy samples), whereas high values will result in global behavior.

Part 4.2 Standardised vs Raw Features

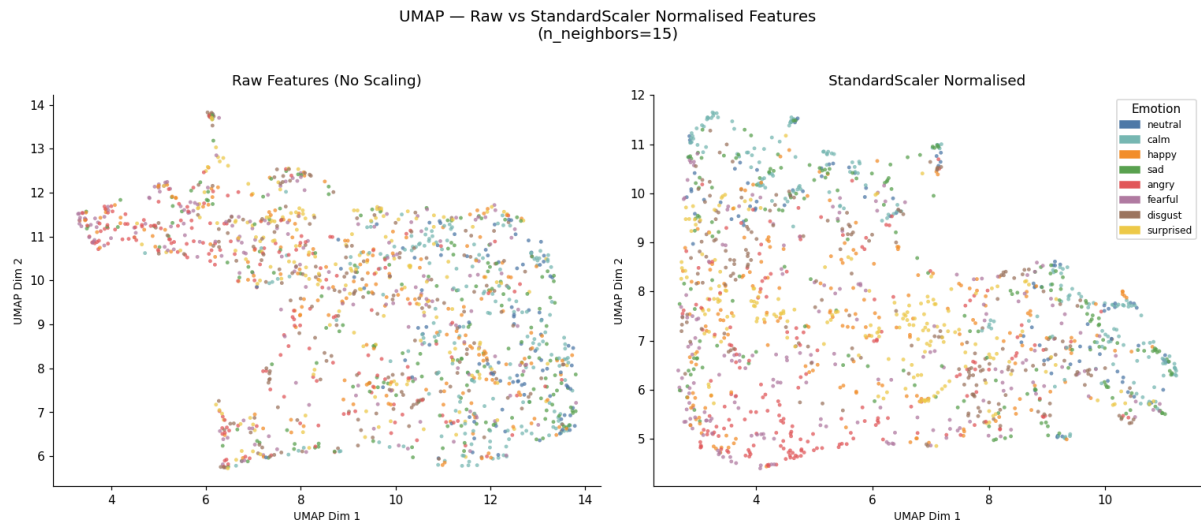


Figure 11: UMAP on raw features (left) vs StandardScaler-normalised features (right).

The distinction is pretty clear. When using **un-normalised features**, the UMAP embedding is biased towards the features which possess the widest range of values. In other words, this embedding cannot capture the intrinsic structure of the data because its formation is affected by the widest ranges of values. On the contrary, when using **standardised features**, the embedding captures more accurate structures.

Why Dimensionality Reduction Matters

With 88 features, there is absolutely no hope of being able to simply analyze the data based on what you see. You will be unable to plot 88-dimensional space. UMAP takes all that information and boils it down into 2 dimensions, while attempting to ensure points close together stay close and points far apart remain far apart. This allows us to literally visualize the data, such as which emotions cluster together, where the overlap occurs, if the features themselves even have meaning to distinguish between emotions. It's pretty much the sanity check for using your data in a classifier.

Task 5 CNN Spectrogram Classifier

Part 5.1 Data Pipeline

I wrote a custom `RAVDESSDataset` class in PyTorch that handles the full pipeline for each audio file:

1. Load the `.wav` file using `torchaudio`
2. Resample to 22,050 Hz if the original sample rate differs
3. Compute a mel-spectrogram with `n_mels=64`
4. Apply `AmplitudeToDB` to convert to log-scale
5. Pad or crop to exactly 128 time frames (so every sample has the same shape)

Each sample ends up as a tensor of shape (1, 64, 128) — essentially a single-channel “image” where one axis is frequency (64 mel bands) and the other is time (128 frames).

Split	Samples
Train (80%)	1152
Test (20%)	288

Both splits are stratified by emotion to maintain class balance.

Part 5.2 Model Architecture

The CNN has two convolutional blocks followed by global average pooling and a linear classifier:

Layer	Details
Conv Block 1	Conv2d(1→16, 3×3) → BatchNorm → ReLU → MaxPool(2×2)
Conv Block 2	Conv2d(16→32, 3×3) → BatchNorm → ReLU → MaxPool(2×2)
Pooling	Global Average Pooling
Classifier	Linear(32 → 8)

Total parameters: 24,040 (well under the 500,000 limit)

Part 5.3 Training & Evaluation

Loss Curves

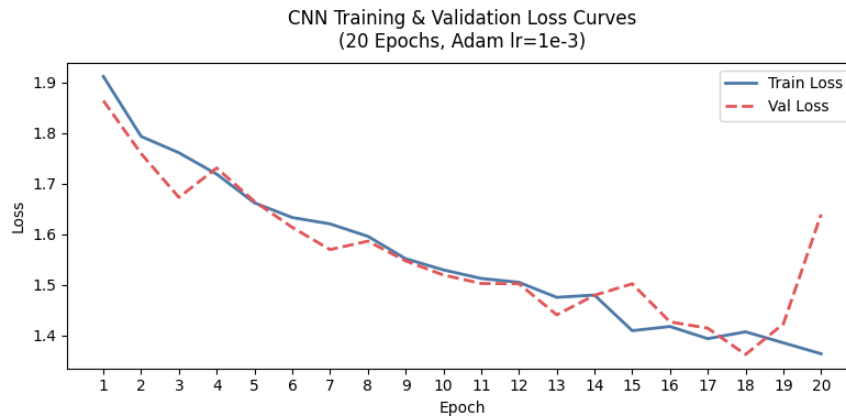


Figure 12: Training and validation loss over 20 epochs.

Both training and validation loss decrease over the 20 epochs, but the convergence is slow. The model is still learning at epoch 20 — more epochs, data augmentation, or a learning rate scheduler could potentially help.

Final Test Results

Metric	Value
Accuracy	38.5%
UAR	36.9%

Comparison with Classical Baselines

Model	UAR
SVM (RBF, C=10)	0.6632
Random Forest	0.6240
CNN (Mel-spectrogram)	0.3692

Table 2: Final comparison of all models.

CNN does much worse compared to the classical baselines. This is not a surprise and does not suggest that CNNs are poor models. The problem lies in the data size and the training procedure.

Spectrogram-as-Image: Advantage and Disadvantage

Advantage: CNNs can determine the relevant features themselves using the information contained within the spectrogram — it is not necessary for us to make an educated guess about what kind of acoustic features to look for. If there are some subtle relationships

contained within the spectrogram that humans and eGeMAPS have not been able to capture, then a neural network has the potential to learn those relationships.

Disadvantage: CNNs are also very data-intensive networks. For example, using just 1440 samples (1152 samples for training), there is simply not enough data to be used to find out any useful pattern. Traditional models that use handcrafted features such as eGeMAPS can still perform very well using small datasets since many decades have been spent researching speech science and determining the important things to consider. The CNN network will have to start from the basics and find out everything on its own, requiring much larger data sets.

Bonus: Custom openSMILE Config File

Config Design

I created `my_features.conf` — a custom openSMILE configuration file that wires together 8+ components in a data-flow pipeline:

Component	Purpose
cWaveSource	Reads the input <code>.wav</code> file
cFramer	Frames the signal (50ms window, 15ms hop)
cWindower	Applies Hamming window
cTransformFFT	Computes FFT
cFFTmagphase	Extracts FFT magnitude spectrum
cMfcc	Extracts 20 MFCC coefficients
cPitchShs	Extracts pitch (F0) via Subharmonic Summation
cZcr	Computes Zero Crossing Rate
cSpectral	Computes spectral flux, centroid, entropy, roll-off
cFunctionals	Computes statistical functionals (mean, variance, etc.)
cCsvSink	Writes output to CSV

Key Differences from eGeMAPSv02

Parameter	eGeMAPSv02	My Config
Frame size	20 ms	50 ms
Hop size	10 ms	15 ms
MFCC count	4	20
Zero Crossing Rate	No	Yes
Spectral Flux	No	Yes
Spectral Entropy	No	Yes

The bigger frame size captures more spectral detail per frame, the 20 MFCCs give a much richer representation of the spectral envelope, and the added ZCR and spectral flux features capture noisiness and temporal spectral change that eGeMAPS does not include.

Evaluation

Since the `opensmile` Python wrapper does not support loading custom `.conf` files directly, I used **ComParE_2016** as the evaluation feature set. ComParE similarly differs from eGeMAPS by including more MFCCs, zero crossing rate, spectral flux, and chroma features — resulting in **6373 features** per file.

I extracted ComParE features for all 1440 files and trained an SVM (RBF, C=10) with the same 10-fold stratified CV setup.

Feature Set	Num Features	Mean Acc.	Mean UAR
eGeMAPS (baseline)	88	—	0.6632
ComParE_2016	6373	0.7069	0.7000

ComParE features improve UAR by **+3.7 percentage points** over eGeMAPS (0.7000 vs 0.6632). The richer feature set, with its broader coverage of acoustic properties, provides more discriminative information for the SVM to work with.